

Project Report: DineFlow Restaurant Management System

Course: Advanced JavaScript Backend Frameworks (Node.js & Express JS) • 5th Semester

Institution: Department of Computer Science, Christ University in partnership with L&T; EduTech

Project Code & Title:	P07 — Multi-Branch Restaurant Management & Table Reservation System
Batch / Section:	Batch 47 (4)
Submission Date:	September 10, 2026
Development Stack:	Node.js, Express.js, MongoDB (Mongoose), JWT, bcrypt, Bootstrap 5

Mandatory Team Details

S.No	Student Name	Roll No. / Reg No.	Department	Section / Batch
1	ALAN R S (Lead)	2247101	Computer Science	Batch 47 (4)
2	Team Member 2	2247102	Computer Science	Batch 47 (4)
3	Team Member 3	2247103	Computer Science	Batch 47 (4)
4	Team Member 4	2247104	Computer Science	Batch 47 (4)

■ Mandatory GitHub Repository Link:

https://github.com/Alan-rs-hub/L-T_Project_Restaurant_management

Live Local Server URL: <http://localhost:5001> | **API Base:** <http://localhost:5001/api>

1. Executive Summary & Problem Statement

DineFlow is an enterprise-grade, full-stack Restaurant Management and Table Reservation platform engineered to streamline restaurant operations across multiple physical branches. Developed with a rigorous MVC backend architecture, the system provides guaranteed table overbooking prevention, automated state-machine driven kitchen order workflows, unalterable server-side itemized billing, and management analytics.

Role-Based Access Control (RBAC): The platform securely supports four distinct roles: *Customer* (bookings, food orders, reviews), *Kitchen Staff* (real-time 3-stage KDS queue), *Branch Manager* (dishes, inventory, sales reports), and *System Administrator* (branch CRUD, table inventory, global analytics).

2. System Architecture & Technology Stack

The project strictly adopts the **Model-View-Controller (MVC)** architectural pattern to ensure clean separation of concerns, high testability, and enterprise-grade maintainability.

Component Layer	Technology / Library	Purpose & Key Responsibility
Backend Runtime	Node.js (v16+) & Express.js (v4.21.0)	RESTful API service, routing, middleware orchestration
Database & ODM	MongoDB & Mongoose (v8.6.0)	Document store, schema validation, compound indexes
Authentication	JWT (jsonwebtoken) & bcryptjs (12 rounds)	Stateless token auth, encrypted password hashing
Input Validation	Joi (v17.13.3) & Custom Middleware	Strict schema validation for all incoming request payloads
Client-Side UI	HTML5, CSS3, JavaScript ES6+, Bootstrap 5	Responsive dark glassmorphism portal with live dynamic feeds
Zero-Config DB	mongodb-memory-server	Automated embedded MongoDB fallback for seamless evaluation

3. Database Design & Mongoose Schemas (7 Models)

The database layer is structured across 7 normalized Mongoose models with referential integrity, compound indexes, and pre-save hooks:

Model Name	Key Fields & Types	Indexes & Constraints
User	name, email, passwordHash, role	unique: email; bcrypt 12-round pre-save hook; stripped on JSON output
Branch	name, address, seatingCapacity, isActive	Unique branch naming; active status filtering
Table	branchId (Ref), tableNumber, capacity, isActive	Compound unique index { branchId: 1, tableNumber: 1 }
MenuItem	branchId (Ref), name, category, price, isAvailable	Category enum (8 types); compound index { branchId: 1, name: 1 }
Reservation	customerId, branchId, tableId, dateTime, duration, partySize, status	Index on tableId+dateTime+status for real-time overlap conflict check
Order	orderNumber, customerId, branchId, items[], billing{}, orderType, status	Unique orderNumber; embedded billing subdocument; status workflow enum
Feedback	orderId (Ref), customerId (Ref), rating (1-5), comment	Compound unique index { orderId: 1 } (1 review per completed order)

4. Implemented Functional Modules (All 13 Modules)

Every module outlined in the assessment specification is fully functional and verified:

#	Module Name	Role	Implementation & Business Logic
1	User Auth & RBAC	All	JWT token creation, bcrypt salted hashing, role guards (Customer, Kitchen, Manager, Admin).
2	Branch Management	Admin	CRUD operations on multi-location restaurant branches with seating capacity tracking.
3	Table Inventory	Admin	Table allocation per branch with capacity limits; duplicate prevention via compound index.
4	Menu Management	Admin, Mgr	Branch-specific catalogs across 8 categories; stock availability toggling and text search.
5	Table Reservation	Customer	Real-time table overlap conflict detection: (start < existEnd && end > existStart).
6	Food Ordering	Customer	Dine-in & Takeaway cart ordering with stock and branch affinity validation.
7	Order Workflow State	Kitchen, Mgr	Strict state transitions: placed → preparing → ready → served/delivered.
8	Kitchen Display (KDS)	Kitchen	3-column real-time KDS board, 10s auto-refresh, and synthesized Web Audio chime.
9	Itemized Billing Engine	System	Server-side deterministic math: Subtotal + 5% Tax + 10% Service Charge = Grand Total.
10	Cancellation Policy	Customer	1-hour notice policy enforced on customer cancellations; admin retains override authority.
11	Customer Order History	Customer	Chronological itemized invoices, real-time status badges, and receipt downloads.
12	Customer Feedback	Customer	1-5 star ratings & reviews allowed exclusively on completed orders; duplicates blocked.
13	Manager Analytics	Manager, Admin	Aggregation pipelines: Revenue by branch, top 5 dishes, peak activity hours, KPI overview.

5. Core Business Rules & Validation Logic

1. Table Double-Booking & Overlap Prevention Engine:

To eliminate race conditions and overbooking, the reservation controller executes an intersection query across all active confirmed bookings for the target table: `start < existEnd && end > existStart`. Overlapping booking requests are rejected with a 409 `RESERVATION_CONFLICT` error.

2. Finite Order State Machine:

Order status progression is governed by a strict transition matrix: `placed` → `[preparing, cancelled]`, `preparing` → `[ready]`, `ready` → `[served, delivered]`. Unauthorized skips or terminal transitions trigger 409 `INVALID_STATUS_TRANSITION`.

3. Reservation Cancellation Policy:

Customers attempting to cancel reservations with less than 60 minutes remaining before the scheduled reservation time are rejected with 409 `CANCELLATION_POLICY`, protecting restaurant table planning.

4. Zero-Trust Server-Side Itemized Billing:

To prevent client-side financial tampering, all item totals, subtotal, tax (5%), service charge (10%), and grand totals are deterministically computed on the server in `utils/calculations.js` from the database menu item prices.

6. Key REST API Endpoint Reference

Endpoint Route	Method	Auth / Access	Summary & Expected Output
<code>/api/auth/register</code>	POST	Public	Registers user; returns JWT token & user profile
<code>/api/auth/login</code>	POST	Public	Authenticates credentials; returns JWT bearer token
<code>/api/branches</code>	GET / POST	Public / Admin	Lists active branches / Creates new branch entity
<code>/api/tables/available</code>	GET	Authenticated	Searches available tables matching date, time, and party size
<code>/api/menu</code>	GET / POST	Public / Mgr, Admin	Catalog search & filtering / Creates menu dish item
<code>/api/reservations</code>	GET / POST	Authenticated	Lists bookings / Books table with overlap conflict check
<code>/api/reservations/:id</code>	DELETE	Customer, Admin	Cancels booking enforcing 1-hour advance notice policy
<code>/api/orders</code>	POST / GET	Authenticated	Places food order with server billing / Lists order history
<code>/api/orders/:id/status</code>	PUT	Kitchen, Admin	Advances order workflow state (<code>placed</code> → <code>preparing</code> → <code>ready</code>)
<code>/api/kitchen/orders</code>	GET	Kitchen, Admin	Real-time active queue for placed, preparing, and ready orders
<code>/api/feedback</code>	POST / GET	Customer	Submits 1-5 star review for completed (<code>served/delivered</code>) orders
<code>/api/manager/reports/sales</code>	GET	Manager, Admin	Aggregated sales revenue, order counts, and avg values by branch

7. Quick Start & Execution Guide

Follow these instructions to run the application locally or in an evaluation environment:

Step 1: Install Dependencies & Run Database Seeder

```
npm install
npm run seed (Populates users, branches, tables, menu items, reservations, orders, and reviews)
```

Step 2: Launch the Application Server

```
npm start (or npm run dev for development mode)
Application will be live at: http://localhost:5001
```

Pre-Configured Demo Test Accounts:

Role	Email Address	Password	Accessible Features & Portals
Admin	admin@dineflow.com	admin123	Full dashboard, branch & table management, sales reports, menu & orders
Manager	manager@dineflow.com	manager123	Analytics, popular dishes, peak hours, menu management, KDS
Kitchen Staff	kitchen@dineflow.com	kitchen123	Real-time 3-stage Kitchen Display System (KDS) board
Customer 1	customer@dineflow.com	customer123	Table reservation booking, food ordering, invoice tracking, feedback
Customer 2	priya@dineflow.com	customer123	Table reservation booking, food ordering, invoice tracking, feedback

8. Testing & Evaluation Verification Summary

1. **Postman Test Suite:** The project includes an exported Postman collection (postman_collection.json) containing over 30 test requests spanning authentication, branch/table inventory, menu catalog queries, table booking conflict checks, order state transitions, and analytics aggregations.
2. **End-to-End Browser Flow:** All 8 web pages (Landing, Login, Register, Menu, Reservations, Orders & Billing, Kitchen Display, Dashboard) were thoroughly validated via automated browser subagents.
3. **Zero-Configuration Database:** The backend features automated embedded MongoDB fallback (mongodb-memory-server) to ensure the evaluators can clone, run, and evaluate the project immediately with zero local MongoDB daemon setup required.

Submitted for Continuous Internal Assessment - 3 (CIA-3)
Department of Computer Science • Christ University • Academic Year 2026
Repository: https://github.com/Alan-rs-hub/L-T_Project_Restaurant_management